

utilizando python + django + mysql criar um projeto que possua crud de clientes e crud de veiculos. o projeto deve se chamar AutoFlow

Vou montar a base completa do projeto Django com MySQL, incluindo os dois módulos de CRUD: clientes e veículos, já organizada para você usar como ponto de partida.

Pensou por 7s

AutoFlow — Projeto Django + MySQL

Abaixo está um projeto completo base para **Python + Django + MySQL** com:

- CRUD de **Clientes**
- CRUD de **Veículos**
- Relacionamento: um cliente pode ter vários veículos
- Estrutura pronta para rodar localmente

1. Estrutura do projeto

```
AutoFlow/  
├─ manage.py  
├─ requirements.txt  
├─ AutoFlow/  
│   ├── __init__.py  
│   ├── settings.py  
│   ├── urls.py  
│   ├── asgi.py  
│   └─ wsgi.py  
└─ clientes/
```

```
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── forms.py
├── views.py
├── urls.py
├── tests.py
├── migrations/
│   └── __init__.py
├── veiculos/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── forms.py
│   ├── views.py
│   ├── urls.py
│   ├── tests.py
│   └── migrations/
│       └── __init__.py
└── templates/
    ├── base.html
    ├── home.html
    ├── clientes/
    │   ├── lista.html
    │   ├── form.html
    │   ├── detalhe.html
    │   └── confirmar_exclusao.html
    └── veiculos/
        ├── lista.html
        ├── form.html
        ├── detalhe.html
        └── confirmar_exclusao.html
```

2. requirements.txt

```
Django>=5.0,<6.0
mysqlclient>=2.2
```

Se tiver dificuldade com `mysqlclient` no Windows, você pode usar `PyMySQL`.

3. Criação do ambiente e instalação

```
python -m venv venv
```

Windows

```
venv\Scripts\activate
```

Linux/Mac

```
source venv/bin/activate
```

Depois:

```
pip install -r requirements.txt
```

4. Criar o projeto Django

```
django-admin startproject AutoFlow .  
python manage.py startapp clientes  
python manage.py startapp veiculos
```

5. Banco MySQL

Crie o banco no MySQL:

```
CREATE DATABASE autoflow CHARACTER SET utf8mb4 COLLATE utf8mb4_ur
```

6. Arquivo AutoFlow/settings.py

```
from pathlib import Path  
  
BASE_DIR = Path(__file__).resolve().parent.parent
```

```
SECRET_KEY = 'django-insecure-troque-esta-chave'
DEBUG = True
ALLOWED_HOSTS = []

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'clientes',
    'veiculos',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'AutoFlow.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTempl
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.mes
            ],
        },
    },
]

WSGI_APPLICATION = 'AutoFlow.wsgi.application'

DATABASES = {
```

```

'default': {
    'ENGINE': 'django.db.backends.mysql',
    'NAME': 'autoflow',
    'USER': 'root',
    'PASSWORD': 'SUA_SENHA',
    'HOST': '127.0.0.1',
    'PORT': '3306',
    'OPTIONS': {
        'init_command': "SET sql_mode='STRICT_TRANS_TABLES'"
    },
}
}

AUTH_PASSWORD_VALIDATORS = [
    {'NAME': 'django.contrib.auth.password_validation.UserAttri
    {'NAME': 'django.contrib.auth.password_validation.MinimumLe
    {'NAME': 'django.contrib.auth.password_validation.CommonPas
    {'NAME': 'django.contrib.auth.password_validation.NumericPa
]

LANGUAGE_CODE = 'pt-br'
TIME_ZONE = 'America/Sao_Paulo'
USE_I18N = True
USE_TZ = True

STATIC_URL = 'static/'

```

7. Arquivo AutoFlow/urls.py

```

from django.contrib import admin
from django.urls import path, include
from django.shortcuts import render

def home(request):
    return render(request, 'home.html')

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', home, name='home'),
    path('clientes/', include('clientes.urls')),

```

```
path('veiculos/', include('veiculos.urls'))),
```

8. App clientes

clientes/models.py

```
from django.db import models

class Cliente(models.Model):
    nome = models.CharField(max_length=150)
    cpf = models.CharField(max_length=14, unique=True)
    email = models.EmailField(unique=True)
    telefone = models.CharField(max_length=20, blank=True, null=True)
    endereco = models.CharField(max_length=255, blank=True, null=True)
    data_cadastro = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['nome']
        verbose_name = 'Cliente'
        verbose_name_plural = 'Clientes'

    def __str__(self):
        return f'{self.nome} - {self.cpf}'
```

clientes/forms.py

```
from django import forms
from .models import Cliente

class ClienteForm(forms.ModelForm):
    class Meta:
        model = Cliente
        fields = ['nome', 'cpf', 'email', 'telefone', 'endereco']
```

clientes/admin.py

```
from django.contrib import admin
from .models import Cliente
```

```
@admin.register(Cliente)
class ClienteAdmin(admin.ModelAdmin):
    list_display = ('id', 'nome', 'cpf', 'email', 'telefone', ' ')
    search_fields = ('nome', 'cpf', 'email')
```

```
class ClienteAdmin(admin.ModelAdmin):
```

```
from django.urls import reverse_lazy
from django.views.generic import ListView, CreateView, UpdateView
from .models import Cliente
from .forms import ClienteForm

class ClienteListView(ListView):
    model = Cliente
    template_name = 'clientes/lista.html'
    context_object_name = 'clientes'

class ClienteCreateView(CreateView):
    model = Cliente
    form_class = ClienteForm
    template_name = 'clientes/form.html'
    success_url = reverse_lazy('clientes_lista')

class ClienteUpdateView(UpdateView):
    model = Cliente
    form_class = ClienteForm
    template_name = 'clientes/form.html'
    success_url = reverse_lazy('clientes_lista')

class ClienteDeleteView(DeleteView):
    model = Cliente
    template_name = 'clientes/confirmar_exclusao.html'
    success_url = reverse_lazy('clientes_lista')

class ClienteDetailView(DetailView):
    model = Cliente
    template_name = 'clientes/detalhe.html'
    context_object_name = 'cliente'
```

clientes/urls.py

```
from django.urls import path
from .views import (
    ClienteListView,
    ClienteCreateView,
    ClienteUpdateView,
    ClienteDeleteView,
```



```

        ClienteDetailView,
    )

urlpatterns = [
    path('', ClienteListView.as_view(), name='clientes_lista'),
    path('novo/', ClienteCreateView.as_view(), name='clientes_novo'),
    path('<int:pk>/', ClienteDetailView.as_view(), name='cliente_detail'),
    path('<int:pk>/editar/', ClienteUpdateView.as_view(), name='cliente_editar'),
    path('<int:pk>/excluir/', ClienteDeleteView.as_view(), name='cliente_excluir')
]

```

9. App veículos

veiculos/models.py

```

from django.db import models
from clientes.models import Cliente

class Veiculo(models.Model):
    cliente = models.ForeignKey(Cliente, on_delete=models.CASCADE)
    marca = models.CharField(max_length=100)
    modelo = models.CharField(max_length=100)
    ano = models.PositiveIntegerField()
    placa = models.CharField(max_length=10, unique=True)
    cor = models.CharField(max_length=50, blank=True, null=True)
    chassi = models.CharField(max_length=30, unique=True)
    data_cadastro = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['marca', 'modelo']
        verbose_name = 'Veículo'
        verbose_name_plural = 'Veículos'

    def __str__(self):
        return f'{self.marca} {self.modelo} - {self.placa}'

```

veiculos/forms.py

```

from django import forms
from .models import Veiculo

```

```
class VeiculoForm(forms.ModelForm):
    class Meta:
        model = Veiculo
        fields = ['cliente', 'marca', 'modelo', 'ano', 'placa',
```

```
from django.contrib import admin
from .models import Veiculo

@admin.register(Veiculo)
class VeiculoAdmin(admin.ModelAdmin):
    list_display = ('id', 'cliente', 'marca', 'modelo', 'ano', 'p
    search_fields = ('marca', 'modelo', 'placa', 'chassi', 'clier
    list_filter = ('marca', 'ano')
```

veiculos/views.py

```
from django.urls import reverse_lazy
from django.views.generic import ListView, CreateView, UpdateVi
from .models import Veiculo
from .forms import VeiculoForm

class VeiculoListView(ListView):
    model = Veiculo
    template_name = 'veiculos/lista.html'
    context_object_name = 'veiculos'

class VeiculoCreateView(CreateView):
    model = Veiculo
    form_class = VeiculoForm
    template_name = 'veiculos/form.html'
    success_url = reverse_lazy('veiculos_lista')

class VeiculoUpdateView(UpdateView):
    model = Veiculo
    form_class = VeiculoForm
    template_name = 'veiculos/form.html'
```

```

        success_url = reverse_lazy('veiculos_lista')

class VeiculoDeleteView(DeleteView):
    model = Veiculo
    template_name = 'veiculos/confirmar_exclusao.html'
    success_url = reverse_lazy('veiculos_lista')

class VeiculoDetailView(DetailView):
    model = Veiculo
    template_name = 'veiculos/detalhe.html'
    context_object_name = 'veiculo'

```

veiculos/urls.py

```

from django.urls import path
from .views import (
    VeiculoListView,
    VeiculoCreateView,
    VeiculoUpdateView,
    VeiculoDeleteView,
    VeiculoDetailView,
)

urlpatterns = [
    path('', VeiculoListView.as_view(), name='veiculos_lista'),
    path('novo/', VeiculoCreateView.as_view(), name='veiculos_novo'),
    path('<int:pk>/', VeiculoDetailView.as_view(), name='veiculos_detalhe'),
    path('<int:pk>/editar/', VeiculoUpdateView.as_view(), name='veiculos_editar'),
    path('<int:pk>/excluir/', VeiculoDeleteView.as_view(), name='veiculos_excluir'),
]

```

10. Templates

templates/base.html

```

<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">

```

```
<meta name="viewport" content="width=device-width, initial-  
<title>AutoFlow</title>  
<style>  
  body {  
    font-family: Arial, sans-serif;  
    margin: 0;  
    padding: 0;  
    background: #f4f6f8;  
  }  
  header {  
    background: #1f2937;  
    color: white;  
    padding: 16px;  
  }  
  nav a {  
    color: white;  
    margin-right: 16px;  
    text-decoration: none;  
    font-weight: bold;  
  }  
  .container {  
    width: 90%;  
    max-width: 1100px;  
    margin: 30px auto;  
    background: white;  
    padding: 24px;  
    border-radius: 8px;  
    box-shadow: 0 2px 10px rgba(0,0,0,0.08);  
  }  
  table {  
    width: 100%;  
    border-collapse: collapse;  
  }  
  table, th, td {  
    border: 1px solid #ddd;  
  }  
  th, td {  
    padding: 10px;  
    text-align: left;  
  }  
  .btn {  
    display: inline-block;  
    padding: 10px 14px;  
    background: #2563eb;  
    color: white;  
    text-decoration: none;
```

```

        border-radius: 6px;
        border: none;
        cursor: pointer;
    }
    .btn-danger {
        background: #dc2626;
    }
    .btn-secondary {
        background: #6b7280;
    }
    form p {
        margin-bottom: 12px;
    }
    input, select {
        width: 100%;
        padding: 10px;
        border: 1px solid #ccc;
        border-radius: 6px;
    }
</style>
</head>
<body>
    <header>
        <h1>AutoFlow</h1>
        <nav>
            <a href="/">Início</a>
            <a href="/clientes/">Clientes</a>
            <a href="/veiculos/">Veículos</a>
            <a href="/admin/">Admin</a>
        </nav>
    </header>

    <div class="container">
        {% block content %}{% endblock %}
    </div>
</body>

```

templates/home.html

```

{% extends 'base.html' %}

{% block content %}
<h2>Bem-vindo ao AutoFlow</h2>
<p>Sistema de cadastro de clientes e veículos.</p>
<a class="btn" href="/clientes/">Gerenciar Clientes</a>

```

```
<a class="btn" href="/veiculos/">Gerenciar Veículos</a>
{% endblock %}
```

templates/clientes/lista.html

```
{% extends 'base.html' %}

{% block content %}
<h2>Clientes</h2>
<a class="btn" href="{% url 'clientes_novo' %}">Novo Cliente</a>
<br><br>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Nome</th>
      <th>CPF</th>
      <th>Email</th>
      <th>Telefone</th>
      <th>Ações</th>
    </tr>
  </thead>
  <tbody>
    {% for cliente in clientes %}
    <tr>
      <td>{{ cliente.id }}</td>
      <td>{{ cliente.nome }}</td>
      <td>{{ cliente.cpf }}</td>
      <td>{{ cliente.email }}</td>
      <td>{{ cliente.telefone }}</td>
      <td>
        <a href="{% url 'clientes_detalhe' cliente.id %}">
        <a href="{% url 'clientes_editar' cliente.id %}">
        <a href="{% url 'clientes_excluir' cliente.id %}">
      </td>
    </tr>
    {% empty %}
    <tr>
      <td colspan="6">Nenhum cliente cadastrado.</td>
    </tr>
    {% endfor %}
  </tbody>
</table>
{% endblock %}
```

templates/clientes/form.html

```
{% extends 'base.html' %}

{% block content %}
<h2>Cadastro de Cliente</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button class="btn" type="submit">Salvar</button>
    <a class="btn btn-secondary" href="{% url 'clientes_lista' %}">
</form>
{% endblock %}
```

templates/clientes/detalhe.html

```
{% extends 'base.html' %}

{% block content %}
<h2>Detalhes do Cliente</h2>
<p><strong>ID:</strong> {{ cliente.id }}</p>
<p><strong>Nome:</strong> {{ cliente.nome }}</p>
<p><strong>CPF:</strong> {{ cliente.cpf }}</p>
<p><strong>Email:</strong> {{ cliente.email }}</p>
<p><strong>Telefone:</strong> {{ cliente.telefone }}</p>
<p><strong>Endereço:</strong> {{ cliente.endereco }}</p>
<p><strong>Data de Cadastro:</strong> {{ cliente.data_cadastro }}</p>
<a class="btn" href="{% url 'clientes_editar' cliente.id %}">Edit
<a class="btn btn-secondary" href="{% url 'clientes_lista' %}">Vc
{% endblock %}
```

templates/clientes/confirmar_exclusao.html

```
{% extends 'base.html' %}

{% block content %}
<h2>Excluir Cliente</h2>
<p>Tem certeza que deseja excluir <strong>{{ object.nome }}</strong>?</p>
<form method="post">
    {% csrf_token %}
    <button class="btn btn-danger" type="submit">Confirmar Excl
    <a class="btn btn-secondary" href="{% url 'clientes_lista' %}">
</form>
```

templates/veiculos/lista.html

```
{% extends 'base.html' %}

{% block content %}
<h2>Veículos</h2>
<a class="btn" href="{% url 'veiculos_novo' %}">Novo Veículo</a>
<br><br>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Cliente</th>
      <th>Marca</th>
      <th>Modelo</th>
      <th>Ano</th>
      <th>Placa</th>
      <th>Ações</th>
    </tr>
  </thead>
  <tbody>
    {% for veiculo in veiculos %}
    <tr>
      <td>{{ veiculo.id }}</td>
      <td>{{ veiculo.cliente.nome }}</td>
      <td>{{ veiculo.marca }}</td>
      <td>{{ veiculo.modelo }}</td>
      <td>{{ veiculo.ano }}</td>
      <td>{{ veiculo.placa }}</td>
      <td>
        <a href="{% url 'veiculos_detalhe' veiculo.id %}">
        <a href="{% url 'veiculos_editar' veiculo.id %}">
        <a href="{% url 'veiculos_excluir' veiculo.id %}">
      </td>
    </tr>
    {% empty %}
    <tr>
      <td colspan="7">Nenhum veículo cadastrado.</td>
    </tr>
    {% endfor %}
  </tbody>
</table>
{% endblock %}
```


templates/veiculos/form.html

```
{% extends 'base.html' %}

{% block content %}
<h2>Cadastro de Veículo</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button class="btn" type="submit">Salvar</button>
    <a class="btn btn-secondary" href="{% url 'veiculos_lista' %}">
</form>
{% endblock %}
```

templates/veiculos/detalhe.html

```
{% extends 'base.html' %}

{% block content %}
<h2>Detalhes do Veículo</h2>
<p><strong>ID:</strong> {{ veiculo.id }}</p>
<p><strong>Cliente:</strong> {{ veiculo.cliente.nome }}</p>
<p><strong>Marca:</strong> {{ veiculo.marca }}</p>
<p><strong>Modelo:</strong> {{ veiculo.modelo }}</p>
<p><strong>Ano:</strong> {{ veiculo.ano }}</p>
<p><strong>Placa:</strong> {{ veiculo.placa }}</p>
<p><strong>Cor:</strong> {{ veiculo.cor }}</p>
<p><strong>Chassi:</strong> {{ veiculo.chassi }}</p>
<p><strong>Data de Cadastro:</strong> {{ veiculo.data_cadastro }}</p>
<a class="btn" href="{% url 'veiculos_editar' veiculo.id %}">Edit
<a class="btn btn-secondary" href="{% url 'veiculos_lista' %}">Vc
{% endblock %}
```

templates/veiculos/confirmar_exclusao.html

```
{% extends 'base.html' %}

{% block content %}
<h2>Excluir Veículo</h2>
<p>Tem certeza que deseja excluir <strong>{{ object.marca }} {{
<form method="post">
    {% csrf_token %}
    <button class="btn btn-danger" type="submit">Confirmar Excl
```

```
<a class="btn btn-secondary" href="{% url 'veiculos_lista' %}">
</form>
```

11. Migrações e execução

```
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Acesse:

- `http://127.0.0.1:8000/`
- `http://127.0.0.1:8000/admin/`
- `http://127.0.0.1:8000/clientes/`
- `http://127.0.0.1:8000/veiculos/`

12. Observações importantes

Relacionamento

- Um **Cliente** possui vários **Veículos**.
- Ao excluir um cliente, os veículos dele também serão excluídos por causa do `on_delete=models.CASCADE`.

Melhorias futuras

Você pode expandir o projeto com:

- Login de usuários
- Busca por placa, CPF ou nome
- Paginação nas listas
- Máscaras para CPF/telefone/placa
- API REST com Django REST Framework

- Dashboard
- Upload de foto do veículo

13. Comandos resumidos

```
python -m venv venv
venv\Scripts\activate
pip install django mysqlclient
AutoFlow Django Mysql Project Copiar Editar Baixar

django-admin startproject AutoFlow .
python manage.py startapp clientes
python manage.py startapp veiculos
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Criei a base do projeto **AutoFlow** com Django + MySQL, incluindo:

- CRUD de **clientes**
- CRUD de **veículos**
- relacionamento entre cliente e veículo
- templates HTML
- models, forms, views e urls
- configuração do MySQL no `settings.py`

O projeto ficou organizado no canvas ao lado para você copiar e usar.

≡ ChatGPT 5.4 Thinking ▾

↑ 📄 ...

```
python -m venv venv
venv\Scripts\activate
pip install django mysqlclient
django-admin startproject AutoFlow .
python manage.py startapp clientes
python manage.py startapp veiculos
python manage.py makemigrations
```

</> Bash

```
python manage.py runserver
```

E no MySQL, crie o banco:

</> SQL

```
CREATE DATABASE autoflow CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

No `settings.py`, ajuste:

</> Python



▶ Executar

```
'NAME': 'autoflow',  
'USER': 'root',  
'PASSWORD': 'SUA_SENHA',  
'HOST': '127.0.0.1',  
'PORT': '3306',
```

Se quiser, no próximo passo eu posso gerar a **versão 2 do AutoFlow**, mais completa, com:

- Bootstrap
- busca por cliente e placa
- paginação
- validações melhores
- layout mais profissional
- dashboard inicial



Você gosta desta personalidade?

